Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 27 de octubre de 2014

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){
 int *A, *C, *F;
 int **B;
 char *D, *E;
 char G;
 int H;
 H = 68;
 G = (char) H;
 A = new int;
 (*A) = H;
 F = A;
 C = F;
 (*A) = 70;
 cout << (*A) << (*C) << (*F) << endl;

 B = &C;
 C = new int;
 (**B) = H + 1;
 cout << (*A) << (*C) << (**B) << endl;

 E = (char*) C;
 D = (char*) (*B);
 cout << (*E) << (*D) << endl;

 (*E) = G;
 cout << (*E) << (*C) << endl;

 F = (int*) D;
 while ( (*C) > 0) {
     (*C)--;
     cout << (*C) << (*D) << (**B) << endl;
     (*F) = (*F) - (**B);
 }

 // liberar la memoria
 return 0;
}
```

2. Implementar para la clase ListaSimplementeEnlazada el método

```
/* pre : la lista tiene al menos tantos elementos como posicionElemento.
 * post: remueve el elemento localizado en la posición posicionElemento de la lista y lo devuelve.
 */
T remover(unsigned int posicionElemento);
```

3. Implementar el método `fabricar` de la clase `FabricaDeVehiculos` a partir de las siguientes especificaciones:

```
class FabricaDeVehiculos {

  public:

    /* post: procesa 'vehiculosPedidos' para determinar que Vehículos se pueden fabricar, consumiendo las Partes desde
     *       'partesDisponibles'. Considera primero la prioridad ascendente e intenta fabricar la mayor cantidad posible.
     *       Devuelve una nueva Lista con los Vehiculos que se pudieron fabricar.
     */
    Lista<Vehiculo*>* fabricar(Lista<Vehiculo*>* vehiculosPedidos, Lista<Parte*>* partesDisponibles);
};
```

```
class Vehiculo {

  public:

    /* post: inicializa el Vehiculo con la prioridad indicada, sin
     *       Partes asociados.
     */
    Vehiculo(short int prioridad);

    /* post: elimina todas las Partes asociadas.
     */
    ~Vehiculo();

    /* post: devuelve la prioridad de fabricación del Vehículo, en el rango
     *       [1, 100]
     */
    short int obtenerPrioridad();

    /* post: devuelve todas las Partes que requiere el Vehículo para su
     *       fabricación.
     */
    Lista<Parte*>* obtenerPartes();

};
```

```
class Parte {

  public:

    /* post: queda inicializado con el nombre y
     *       la cantidad indicados.
     */
    Parte(string nombre, int cantidad);

    ~Parte();

    /* post: devuelve el nombre que la distingue
     */
    string obtenerNombre();

    /* post: devuelve la cantidad de unidades.
     */
    unsigned int obtenerCantidad();

};
```

4. Diseñar la especificación e implementar un TDA que modele el juego del **Quemado**. Debe proveer operaciones para:

- crearlo a partir de la cantidad de jugadores que cada equipo tendrá: compiten dos equipos, el ROJO y el AZUL.
- quemar un jugador de un equipo: el equipo se queda con un jugador menos.
- suspender un jugador de un equipo: el jugador cometió una falta y temporalmente queda fuera del equipo.
- activar jugador de un equipo: el jugador que antes fue suspendido vuelve a jugar.
- saber si el juego terminó: el juego termina cuando un equipo gana.
- conocer el equipo ganador: un equipo gana el juego cuando el otro equipo se quedo sin jugadores. Si hay jugadores suspendidos, pero no activos, el juego queda en espera hasta activar al menos un jugador.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas